МИНОБРНАУКИ РОССИИ
Санкт-Петербургский государственный
электротехнический университет
«ЛЭТИ» им. В.И. Ульянова (Ленина)
Кафедра МОЭВМ

ОТЧЕТ

по практической работе № 2

по дисциплине «Системы параллельной обработки данных»

Тема: «Пляжный волейбол»

Студент гр. 2304 _____ Пархоменко М. С.
подпись

Преподаватель _____ Татаринов Ю. С.
подпись

Санкт-Петербург
2026

1 Цель работы

Изучить обмен сообщениями средствами MPI, реализовать случайный выбор процесса-адресата и исследовать работу параллельной программы при различном числе процессов и различном размере сообщения.

2 Формулировка задания

Вариант 13. Пляжный волейбол. Процесс 0 генерирует сообщение и посылает его любому другому процессу группы. Процесс-приёмник передаёт сообщение дальше. Выбор процесса-адресата осуществляется случайным образом.

3 Описание алгоритма

В программе участвуют p MPI-процессов с рангами от 0 до $p - 1$, причём для выполнения обмена необходимо $p \geq 2$. После инициализации MPI процесс 0 формирует исходное сообщение и случайным образом выбирает первый процесс-получатель среди процессов с рангами от 1 до $p - 1$. Остальные процессы ожидают поступления сообщения.

Получив сообщение, процесс определяет ранг следующего адресата. Адресат выбирается из всех процессов группы, кроме самого отправителя, поэтому сообщение не может быть немедленно послано процессом самому себе. Затем полученное сообщение передаётся выбранному процессу, который повторяет те же действия. Таким образом, в каждый момент обмена один процесс владеет сообщением, а последовательность его перемещений между процессами заранее не определена.

В качестве сообщения используется целое число, обозначающее номер передачи. Процесс 0 начинает обмен со значения 1. Каждый следующий получатель увеличивает число на единицу перед дальнейшей отправкой. В текущей версии игра ограничена десятью передачами.

Обычное сообщение передаётся с тегом TAG_BALL. После получения сообщения с номером 10 процесс не передаёт мяч дальше, а рассылает всем остальным процессам управляющее сообщение с тегом TAG_STOP. Получив этот тег, процесс выходит из цикла приёма. Такой протокол позволяет завершить не только последний процесс-получатель, но и процессы, ожидающие сообщение в MPI_Recv.

Случайный адресат выбирается повторной генерацией числа в диапазоне $[0, p - 1]$ до тех пор, пока полученный ранг не будет отличаться от ранга отправителя. Генератор псевдослучайных чисел каждого процесса получает начальное значение, зависящее от текущего времени и ранга процесса.

4 Реализация и функциональная проверка

Программа реализована на языке C с использованием MPI. Полный листинг текущей версии приведён в приложении А. Для сборки в конфигурацию CMake добавлена цель volleyball. Используется стандарт C11, библиотека MPI::MPI_C и предупреждения компилятора -Wall, -Wextra, -Wpedantic. Сборка выполнена без предупреждений.

После начальной отправки все процессы, включая процесс 0, входят в общий цикл приёма. Вызов MPI_Recv использует MPI_ANY_SOURCE и MPI_ANY_TAG. Фактический отправитель и тег определяются по структуре MPI_Status. Различение обычных и управляющих сообщений выполняется по тегам TAG_BALL и TAG_STOP.

Для функциональной проверки выполнены запуски на двух, четырёх и восьми процессах. Во всех случаях программа выполнила передачи с номерами от 1 до 10, не выбирала текущий процесс в качестве адресата и завершилась с нулевым кодом возврата. Результаты приведены в таблице 1.

Таблица 1 – Результаты функциональной проверки

Число процессов	Число передач	Сообщений остановки	Результат
2	10	1	Успешно
4	10	3	Успешно
8	10	7	Успешно

В одном из запусков на восьми процессах получена последовательность адресатов

$$0 \to 7 \to 5 \to 2 \to 3 \to 0 \to 3 \to 7 \to 4 \to 2 \to 4.$$

Она содержит десять передач, причём соседние ранги различаются. При повторном запуске последовательность может измениться. Строки диагности-

ческого вывода разных процессов также могут отображаться не в порядке номеров передач, поскольку MPI не задаёт общий порядок вывода в стандартный поток.

ПРИЛОЖЕНИЕ А

Листинг программы «Пляжный волейбол»

```
// Автор: Марк Пархоменко
// Дата: 04.11.2026

#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#include <mpi.h>

int choose_dest(int rank, int process_count)
{
   int dest;

   do
   {
      dest = rand() % process_count;
   }
   while (dest == rank);

   return dest;
}

int main(int argc, char *argv[])
{
   enum
```

```
{
   TAG_BALL = 1,
   TAG_STOP = 2
};

int rank;
int process_count;
int ball;
int const max_passes = 10;

MPI_Init(&argc, &argv);

MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &process_count);

srand((unsigned int)time(NULL) + (unsigned int)rank);

if (process_count < 2)
{
   printf("Must have 2 processes\n");
   MPI_Finalize();
   return 0;
}

if (rank == 0)
{
   ball = 1;
   int dest = choose_dest(rank, process_count);
```

```
    MPI_Send(
        &ball,
        1,
        MPI_INT,
        dest,
        TAG_BALL,
        MPI_COMM_WORLD
    );
    printf("rank %d, send ball to rank %d\n",
            rank, dest);

}

bool running = true;

while(running)
{
    int received_msg;
    MPI_Status status;
    MPI_Recv(
        &received_msg,
        1,
        MPI_INT,
        MPI_ANY_SOURCE,
        MPI_ANY_TAG,
        MPI_COMM_WORLD,
        &status
    );
```

```
if (status.MPI_TAG == TAG_BALL)
{
   printf("Sender %d Receiver %d ball %d\n",
         status.MPI_SOURCE, rank, received_msg);

   if (received_msg == max_passes)
   {
      for (int i = 0; i < process_count; i++)
      {
         if (i == rank)
            continue;
         MPI_Send(
            &received_msg,
            1,
            MPI_INT,
            i,
            TAG_STOP,
            MPI_COMM_WORLD
         );
      }
      running = false;

   }
   else
   {
      received_msg++;
      int dest = choose_dest(rank, process_count);
```

```
            MPI_Send(
                &received_msg,
                1,
                MPI_INT,
                dest,
                TAG_BALL,
                MPI_COMM_WORLD
            );
        }

    }
    else if (status.MPI_TAG == TAG_STOP)
    {
        printf("The game was stopped, MPI_TAG %d\n",
                status.MPI_TAG);
        running = false;
    }
  }

  MPI_Finalize();

  return 0;
}
```